Discrete Diffusion Language Modeling for Structured Text Generation

Varesh Patel Urvi Desai Aparajita Sarkar

Abstract

Traditional language models generate text autoregressively, conditioning each token strictly on its previously generated tokens. This project investigates an alternative paradigm - *discrete diffusion* - which treats text generation as a parallel denoising process over the entire sequence simultaneously. We implemented and trained three architecturally distinct discrete diffusion systems from scratch. The first DiffuLLM is a custom Bidirectional Transformer trained with a D3PM-style masked reconstruction loss and a domain-specific BPE tokenizer, trained for 1.2 million steps. The second and third (SEDD-SE and SEDD-KL) both implement the Score Entropy Discrete Diffusion (SEDD) framework on the same architecture, but differ in their training objective: SEDD-SE uses the original score entropy loss from Lou et al. (2023), while SEDD-KL replaces it with a tractable ELBO-based KL divergence loss. All three models were trained on cooking recipe corpora, which exhibit a highly structured, predictable distribution (Title → Ingredients → Instructions). Our results show that (1) DiffuLLM reliably generates coherent, ingredient-grounded cooking instructions at short-to-medium lengths after 500k+ steps; (2) SEDD-SE generates coherent recipe text with correct structure after 100k steps; (3) SEDD-KL achieves lower absolute loss than SEDD-SE at the same step count yet produces dramatically worse samples, confirming a key theoretical prediction about the mode-covering gradient pathology of the KL objective near $t = 0$. We document these results in detail and connect all observations to the mathematical assumptions of the algorithms.

1 Introduction and Motivation

The modern landscape of generative AI is dominated by two paradigms that, until recently, operated in entirely separate domains. *Autoregressive Transformers* (GPT, LLaMA) generate discrete text sequences left-to-right, factoring the joint distribution as a product of conditionals:

$$p(x) = \prod_i p(x_i \mid x_{<i}).$$

Continuous diffusion models (DALL-E, Stable Diffusion) generate images and audio by iteratively denoising a signal drawn from a Gaussian, simultaneously refining the entire output at every step.

The central research question of this project is: *Can diffusion, that has revolutionized continuous data generation, be effectively adapted to the inherently discrete, categorical domain of natural language?*

This question is not merely academic. The autoregressive factorization imposes a strict causal constraint that may be suboptimal for text with strong global dependencies. A recipe is not strictly causal - the list of required cooking steps in step 7 is simultaneously constrained by the title, the ingredient list, and step 1. An autoregressive model generating step 6 must represent all prior context through its key-value cache; a bidirectional parallel model can condition directly on all available context at every denoising step.

2 Background: Discrete Diffusion Models

2.1 Continuous vs. Discrete Diffusion

We adopt the *Discrete Denoising Diffusion Probabilistic Model* (D3PM; Austin et al., 2021) family, which defines the forward process as a Markov chain over categorical distributions governed by a transition matrix Q_t:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}\big(x_t;\ Q_t \cdot e_{x_{t-1}}\big).$$

For continuous-time variants (SEDD), the Markov chain is characterized by a rate matrix Q where $Q_{ij} \geq 0$ for $i \neq j$ is the instantaneous transition rate, and the solution is $p(x_t \mid x_0 = i) = (e^{tQ})_{i,\cdot}$ - the matrix exponential. The dynamics satisfy the Kolmogorov forward equation:

$$\frac{d}{dt} p(x_t) = p(x_t)\, Q.$$

2.2 The Absorbing State Graph

All three of our models use the absorbing (masking) graph, where each token can only transition to a special `[MASK]` state during the forward process. The per-position transition probability at noise level σ is:

$$\begin{aligned} q(x_t = [\mathrm{MASK}] \mid x_0 = i) &= 1 - e^{-\sigma}, \\ q(x_t = i \mid x_0 = i) &= e^{-\sigma}. \end{aligned}$$

This is mathematically equivalent to independently masking each token with probability $1 - e^{-\sigma}$ - the same corruption used in BERT's masked language modeling, but now embedded in a principled probabilistic framework with a continuous noise schedule. At $\sigma = 0$ tokens are fully intact; at $\sigma \to \infty$ everything is masked.

Since only masked positions carry uncertainty, the KL divergence between the true posterior and the model's distribution is nonzero only at masked positions. This is the foundational insight exploited differently by our three training objectives.

2.3 Score Entropy Discrete Diffusion (SEDD)

The SEDD framework (Lou et al., 2023) adapts continuous-time score matching to discrete spaces. Instead of learning $\nabla \log p(x_t)$, SEDD learns a *concrete score* - a ratio:

$$s_\theta(x_t, \sigma)[y] \ \approx\ \frac{p_t(y)}{p_t(x_t)}$$

for each possible token identity y. The score entropy loss is derived as a cross-entropy-like objective over these ratios:

$$\mathcal{L}_{\mathrm{SE}} = \mathbb{E}_{t,x_0,x_t}\Big[\dot\sigma(t) \cdot \mathbf{1}[x_t = [\mathrm{MASK}]] \cdot \Big(\textstyle\sum_{j \neq [\mathrm{MASK}]} e^{s_\theta[j]} \ -\ \frac{s_\theta[x_0]}{e^\sigma - 1} \ +\ C(\sigma)\Big)\Big]$$

where $C(\sigma) = \frac{1}{e^\sigma - 1}\Big(\log \frac{1}{e^\sigma - 1} - 1\Big)$ is a constant in θ. The first term penalizes mass placed anywhere, the second rewards high score on the correct token, and the $\dot\sigma(t)$ weighting concentrates training signal at noise levels where the schedule changes fastest.

3 System A: DiffuLLM

Each recipe is formatted with structured delimiters:

```
<RECIPE_START> <TITLE_START> {title} <TITLE_END>
<INPUT_START> {ingredient_1} <NEXT_INPUT> ... <INPUT_END>
<INSTR_START> {step_1} <NEXT_INSTR> {step_2} ... <INSTR_END>
<RECIPE_END>
```

At inference, providing the title and ingredients as a fixed prefix then unmasking the remaining positions implements conditional generation.

3.1 Model Architecture

The core model is a `D3PMTransformer`: a bidirectional encoder-only Transformer with no causal mask) - every token attends to every other token at every layer. This bidirectional attention is necessary because the denoising objective requires predicting the original token at a masked position conditioned on all visible context, regardless of position.

Adaptive Layer Normalization (AdaLN / DiT-style). A sinusoidal time embedding $\tau(t)$ is projected through a learned MLP and used to modulate the shift, scale, and gate parameters of every layer norm and sublayer:

$$h \leftarrow h + \gamma_{\text{gate}} \cdot \text{Sublayer}\big(\text{LayerNorm}(h) \cdot (1 + \gamma_{\text{scale}}) + \gamma_{\text{shift}}\big)$$

where $\gamma_{\text{gate}}, \gamma_{\text{scale}}, \gamma_{\text{shift}}$ are linear projections of $\tau(t)$. AdaLN output projections are zero-initialized so every block acts as an identity function at the start of training.

Configuration (`recipe_poc_2day` profile):

Hyperparameter	Value
d_{model}	512
Layers	8
Attention heads	8
FFN dimension	2048
Sequence length	256
Total parameters	≈50M

3.2 Forward Process and Noise Schedule

At timestep $t \in \{1, \ldots, T\}$ with $T = 256$, each real token is independently masked with probability:

$$p_{\text{mask}}(t) = 1 - \cos\Big(\frac{t}{T} \cdot \frac{\pi}{2}\Big) \quad \text{[cosine schedule]}.$$

This starts near zero masking at $t = 1$ and reaches near-total masking at $t = T$, with an accelerating rate in the middle to concentrate training budget where the denoising task is hardest.

3.3 Training Objective

Masked-only cross-entropy: the loss penalizes only positions that were actively masked:

$$\mathcal{L}_{\text{D3PM}} = -\frac{1}{|M|} \sum_{i \in M} \log p_\theta\big(x_0^{(i)} \mid x_t, t\big)$$

where $M = \{i : x_t^{(i)} = [\text{MASK}]\}$. Unmasked positions are ignored, focusing capacity entirely on the denoising task.

Logit-normal timestep sampling:

$$u \sim \text{Sigmoid}\big(\mathcal{N}(0,1)\big), \qquad t = \lfloor u(T-1) \rfloor + 1.$$

This concentrates training budget at mid-noise levels where the denoising task is most informative.

3.4 Inference

Starting from a fully `[MASK]` sequence (with optional unmasked prefix), we iteratively fix positions by confidence:

1. Run forward pass, apply top-k filtering and temperature scaling.
2. Compute $n_{\text{fix}}(t)$ = the number of positions that should be fixed by step $t-1$ per the cosine schedule.
3. Among unfixed positions, select the n_{fix} positions with highest confidence (max predicted probability), sample their tokens, permanently fix them.
4. Repeat until all positions are fixed.

4 System B & C: SEDD-SE and SEDD-KL

Systems B and C share an identical architecture, the SEDD DiT-style transformer, but differ in their training objective.

4.1 Shared Architecture

Component	Specification
Hidden size	768
Attention heads	12
Transformer blocks	12
Context length	1,024 tokens
Tokenizer	BPE (50,257 + 1 `[MASK]` = 50,258)
Total parameters	≈169.6M

Timestep conditioning: The noise level σ is embedded via sinusoidal encoding → 2-layer MLP → 128-dim conditioning vector. Each transformer block uses adaptive LayerNorm (adaLN) identical in spirit to DiT (Peebles & Xie, 2022). adaLN parameters are zero-initialized for training stability.

Positional encoding: Rotary embeddings (RoPE) rather than learned absolute positions, allowing generalization to sequence lengths not seen during training.

Attention: Non-causal bidirectional self-attention - every position attends to every other position. Required for parallel denoising.

Output scaling: The raw logits $o_\theta(x_t, \sigma)$ are transformed to log-scores as:

$$s_\theta[j] = o_\theta[j] - \log(e^\sigma - 1) - \log(|V| - 1).$$

This normalization centers log-scores around zero at initialization.

EMA: An Exponential Moving Average (decay = 0.9999) of model parameters is maintained throughout training. Only EMA weights are used at sampling time.

4.2 Shared Noise Schedule

Both systems use the **log-linear** schedule in continuous time:

$$\sigma(t) = -\log\big(1-(1-\varepsilon)t\big), \qquad \varepsilon = 10^{-3},$$

$$\dot{\sigma}(t) \equiv \frac{d\sigma}{dt} = \frac{1-\varepsilon}{1-(1-\varepsilon)t}.$$

The masking probability $1 - e^{-\sigma(t)}$ increases approximately linearly in t, interpolating from ≈ 0 at $t = 0$ to ≈ 1 at $t = 1$.

4.3 System B: Score Entropy Loss (SEDD-SE)

System B uses the original SEDD objective exactly as formulated in Lou et al. (2023), described in Section 2.3. The loss provides gradient signal across all possible token identities at every masked position, making it a rich signal in log-ratio space.

4.4 System C: ELBO-Based KL Loss (SEDD-KL)

System C is the key novel contribution. We derive and implement a tractable ELBO-based KL divergence loss for the absorbing graph - a direct upper bound on $-\log p_\theta(x_0)$.

Derivation

Starting from the ELBO:

$$\log p_\theta(x_0) \;\geq\; \mathbb{E}_q\left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)}\right] \;\equiv\; \text{ELBO}.$$

In the continuous-time limit, the ELBO decomposes into per-step KL terms:

$$-\text{ELBO} = \mathbb{E}_{t\sim\text{Unif}(0,1)}\left[\dot{\sigma}(t)\cdot \mathbb{E}_{x_0,x_t}\left[\sum_\ell \text{KL}\Big(q(x_{t-dt}^\ell \mid x_t^\ell, x_0^\ell)\,\|\,p_\theta(x_{t-dt}^\ell \mid x_t^\ell)\Big)\right]\right] + \text{const}.$$

Deriving the forward posterior. For a masked position ($x_t = [\text{MASK}]$) with true token $x_0 = i$:

$$q_{\text{unmask}} \equiv q(x_{t-dt} = x_0 \mid x_t = [\text{MASK}], x_0) = \frac{\dot{\sigma}(t)\,dt}{e^\sigma - 1},$$
$$q_{\text{mask}} \equiv q(x_{t-dt} = [\text{MASK}] \mid x_t = [\text{MASK}], x_0) = 1 - q_{\text{unmask}}.$$

The quantity $\dot{\sigma}(t)/(e^{\sigma(t)}-1) \approx 1/t$ for small ε. The posterior is a **2-point distribution** over $\{x_0, [\text{MASK}]\}$ - the absorbing forward process only masks tokens, never substitutes them.

The KL at each masked position:

$$\text{KL}(q\|p_\theta) = q_{\text{unmask}}\,\log\frac{q_{\text{unmask}}}{p_\theta(x_0)} + q_{\text{mask}}\,\log\frac{q_{\text{mask}}}{p_\theta([\text{MASK}])}.$$

Full loss:

$$\mathcal{L}_{\text{KL}} = \mathbb{E}_{t,x_0,x_t}\Big[\sum_{i:\,x_t^{(i)}=[\text{MASK}]} \text{KL}_i(q\|p_\theta)\Big].$$

4.5 Theoretical Comparison: SE vs. KL

Property	Score Entropy	KL / ELBO
Divergence minimized	Score entropy divergence	$\mathrm{KL}(q\|p_\theta)$ at each step
Relation to $\log p$	Indirect surrogate	Direct ELBO lower bound
Mode behavior	Mode-seeking tendency	Mode-covering tendency
Gradient weighting	$\propto \dot{\sigma}(t)$	$\propto \dot{\sigma}(t)/(e^{\sigma}-1) \approx 1/t$
Gradient variance near $t=0$	Moderate	Very high
Perplexity interpretable	No	Yes

4.6 Sampling Algorithm

Both systems use predictor-corrector (PC) sampling:

- **Euler predictor**: Discretizes the reverse CTMC. At each step, masked tokens may be unmasked according to the model's predicted scores.
- **Analytic predictor**: Computes an exact categorical posterior using the staggered score.
- **Final denoiser step**: At $t \to 0$, snaps remaining masked tokens to the most probable unmasked tokens.

5 Experimental Setup

5.1 Datasets

System A (DiffuLLM): `B2111797/recipenlg-text-256` (Hugging Face)-500,000 examples from the RecipeNLG corpus, pre-chunked to 256 tokens.

Systems B & C (SEDD): `corbt/all-recipes` (Hugging Face)-structured cooking recipes tokenized with GPT-2 BPE and chunked into 1,024-token blocks.

5.2 Distribution Assumption

We explicitly assume both datasets follow a strong structural distribution: every recipe conforms to Title → Ingredients → Instructions. This assumption is why we expect discrete diffusion to succeed here - bidirectional attention can simultaneously leverage the title and ingredients when generating instructions.

6 Results

All 3 models can generate don't-need-to-squint-for-it-to-look-like English.

6.1 System A: DiffuLLM Results

Training Dynamics

Best validation loss checkpoint captured at **step 503,000**. Further checkpoints at steps 654k, 1172k, and 1200k were captured. Later checkpoints exhibit regression in generation quality despite lower training loss - a sign of overfitting.

Qualitative Results

Example 1: Baked Macaroni and Cheese (checkpoint 503k - Score 17/20)

```
Instructions:
1. In saucepan in a minimum of salted water, cook noodles until tender
2. Drain and set aside
3. In same saucepan, melt butter, stir in flour
4. Gradually add milk, stirring constantly
5. Bring to a boil
6. Stir in cheese, salt and pepper
7. Add cooked noodles
8. Cook until hot and cheese is melted
9. Pour into greased casserole dish
10. Top with bread crumbs
```

Textbook béchamel technique. Correct roux order (butter + flour before milk), all 8 provided ingredients used.

Example 2: Meatloaf (checkpoint 503k - Score 16/20)

```
Instructions:
1. Cook the rice in salted water until just tender (about 15 minutes)
2. Drain
3. Mix the meat mixture with the remaining ingredients
4. Shape into a loaf
5. Bake in a 350 degree oven for 1 hour.
```

Failure Modes at Longer Contexts

At length ≥ 500 tokens and later checkpoints (> 600k steps):

1. **Special token leakage:** Raw control tokens (`<NEXT_INSTR>`) appear verbatim in decoded text.
2. **Token ID drift:** Raw integer token IDs appear past the recipe boundary.
3. **Hallucinated structure:** Structurally plausible but semantically incoherent phrases.
4. **Repetition:** Loop patterns at length ≥ 500, indicating loss of global position tracking.

Checkpoint comparison (Baked Mac and Cheese, 4 samples each):

Checkpoint	Mean Quality Score
best_step503000	$\approx 17/20$
best_step1172000	$\approx 12/20$
final_step1200000	$\approx 12/20$

6.2 Systems B & C: SEDD Results

Run	Loss type	Steps	Train loss	Eval loss
SEDD-SE	Score Entropy	96,750	1,056	1,499
SEDD-KL	KL / ELBO	100,000	663	272

Caveat: The two losses are measured in different units (ratio-space vs. nats per masked token per timestep). Absolute magnitudes are not directly comparable.

Evaluation Loss at Key Steps

Step	SEDD-SE eval	SEDD-KL eval
0	1.077×10^4	6.918×10^3
10,000	1.812×10^3	8.326×10^2
20,000	2.188×10^3	2.110×10^3
50,000	2.436×10^3	1.032×10^3
100,000	1.091×10^3	2.72×10^2

Qualitative Sample Quality at Step 100k

SEDD-SE (step 100k):

```
Spinach Pasta With Parmigiana Sauce

Ingredients:
- 1 16 Oz. box Ziti
- 2 rounded Tablespoons Lemon Zest
- Sea Salt And Pepper
- 1/2 cup Olive Oil
- 2 cups Milk
- 4 ounces, weight Fried Onion
- 1-1/2 cups Diced Bell Pepper (an
- 1 cup each Diced Bell Pepper)
- 1 clove Garlic, minced
- 1 Tablespoon Butter tablespoons Milk
- 4 ounces Parmesan Cheese, Grated
- 1 large Egg, Lightly Beaten

Directions:
- Heat oven 350 degrees.
- Heat large pot of milk over medium heat. Add lemon , olive oil, salt
    and pepper. Bring mixture to a boil and add spinach.
- Cook linguine according to pkg. chicken or chicken is al dente.
   Remove from heat. Saute chopped bell pepper &onion in butter in the
    skillet or a large skillet.
- Mix 2 1/2 cups of sauce +crumbish together. Top with beaten eggs.
   Bake for 30 minutes.<|endoftext|>
```

SEDD-KL (step 100k):

```
Macaroni and Cheeseottage
er BIngredients Cheese cooking 8 1 Veget tablespoons 1i-aronaron
   Season the pastegan.ah packagei4- Bestaronounce-
 for salt it pepper p of Place c 1 cheese 1
 c until flakes oz--.
 and extract cup
 Rinarella cupender 1- groundheddar the 2 Pour Put- Add 1 tsp ham
    forheat red. is T choppedrig bacon, 1 evenlyini well:
s red- driedaronarellaoon Cheese apart And 1 Set. spat it Recipe
   through with; covered cup milk
 spread completely) to,I) pasta cookies add- bringinkle- two done will
    , in boiling 3 a- in, a (- refrigerator
```

Score entropy samples exhibit coherent sentence structure and plausible recipe vocabulary. The KL samples are largely incoherent despite the KL run achieving substantially lower absolute loss.

7 Analysis and Discussion

7.1 Connecting to Course Concepts

KL Divergence and Interpretability. System C's loss is ultimately an attempt to bring interpretation to the field of language modelling by making an objective that approximates the KL divergence of the distributions and also make it more interpretable for perplexity.

The Markov Assumption. All three systems assume $p_\theta(x_{t-1} \mid x_t)$ conditions only on x_t, not the full trajectory. Over long sequences, this creates the repetition and drift observed in DiffuLLM.

Generalization vs. Overfitting. The DiffuLLM checkpoint comparison reveals a canonical overfitting trajectory: training loss continues to decrease past 503k, but generation quality peaks early and degrades. The model memorized specific token sequences, particularly structural endings, rather than learning the underlying distribution.

Bias-Variance Tradeoff in the KL Loss. At small t (near the data), KL gradients are high magnitude but cover very few positions (low bias, high variance). At large t (heavily masked), gradients cover many positions but carry weak signal per position. Score entropy's $\dot{\sigma}(t)$ weighting provides a more uniform tradeoff.

7.2 Analyzing Negative Results

Where does DiffuLLM break down?

Two mechanisms cause structural token leakage at long context:

1. The Markov assumption in action. The denoising process at step t conditions only on the current masked sequence x_t. Over 700 tokens, information about global structural position must be inferred from the pattern of already-unmasked tokens. When many positions are fixed, remaining uncertain positions near sequence boundaries have insufficient context.
2. Global context tracking without forced causality. Autoregressive models maintain a continuous KV cache encoding full left context. Our bidirectional model attends to all tokens simultaneously but has no mechanism forcing it to track progress along the sequence, causing repetition and position amnesia beyond the 256-token training context.

Why did SEDD-KL fail despite lower loss?

The KL loss weighting near $t \to 0$ is $\approx 1/t$. At $t \approx \varepsilon = 10^{-3}$, each masked position contributes a gradient weight $\approx 1000\times$ larger than at $t = 1$. With batch size 8 and $\approx 0.1\%$ of positions masked at small t, the KL gradient is dominated by ≈ 1 token per sequence - a single-sample estimate with enormous variance. The optimizer exploited easy large-t losses while failing to learn fine-grained denoising near $t = 0$.

The lower absolute KL loss (272 vs. 1,056) reflects that the KL loss measures something different, ie, nats per token in probability space, not that the model is better.

7.3 Comparative Summary: All Three Systems

Stepping back from the individual systems, what does this project actually tell us about discrete diffusion for structured text?

The three models we built are not cleanly comparable. They differ in size, training steps, tokenizer, and even what their loss functions are measuring. However, that unevenness is itself informative. DiffuLLM is the smallest and simplest. It reaches genuine coherence. Recipes produced at checkpoint 503k are grounded in their ingredients, follow logical preparation order,

and handle subtle culinary logic (aromatics before sauce, cream added last) correctly. Its weakness is equally fundamental: the Markov denoising process has no causal state-tracker, so beyond ≈500 tokens the model loses track of where it is in the recipe structure and starts repeating phrases or leaking structural delimiter tokens.

SEDD-SE and SEDD-KL share everything except their loss function. They were trained to ≈100k steps, roughly 8% of DiffuLLM's step count, although at larger batch sizes. At this level, SEDD-SE produces partially coherent recipe fragments: the structure is starting to emerge, and the outputs much more coherent text that knows *what kind of thing* it is supposed to be writing. SEDD-KL, despite posting a substantially lower absolute loss (272 vs. 1,056 in their respective units), generates incoherent fragments - high gradient variance near $t \to 0$ destabilizes the fine-grained denoising that coherent output requires. Lower loss did not mean better model; the two objectives are not measuring the same thing.

The honest takeaway is that DiffuLLM's strong results reflect compute and engineering iteration more than an intrinsic advantage of its simpler objective. Given the same 1M+ steps and multi-GPU resources, the SEDD architecture - with its longer context, larger vocabulary, and RoPE positional embeddings, would likely surpass it. What this project surfaces is something more practically useful than a clean ranking: discrete diffusion works, its failure modes are mathematically predictable rather than arbitrary, and the choice of loss function has consequences that only become visible once training has run long enough for the optimization dynamics to dominate.

For reference, the key design choices across all three systems are summarized in Table 1.

Table 1: Design comparison across all three systems.

Dimension	DiffuLLM (A)	SEDD-SE (B)	SEDD-KL (C)
Loss objective	Masked XE	Score entropy	KL/ELBO
Relation to $\log p$	Indirect	Indirect surrogate	Direct lower bound
Noise schedule	Cosine T=256	Log-linear (cts.)	Log-linear (cts.)
Context length	256 tokens	1,024 tokens	1,024 tokens
Model size	≈50M	≈169.6M	≈169.6M
Training steps	1.2M	≈96,750	≈100,000
Inference	Conf. unmasking	Euler PC + Den.	Euler PC + Den.
EMA	No	Yes (0.9999)	Yes (0.9999)
Best sample quality	Excellent (503k)	Partial coherence	Incoherent at 100k
Primary strength	Structured recipes	Arch. sound	Interpretable obj.
Primary limitation	Token leakage >500	Needs more train	Grad. var. at small t

The training budget gap is significant. DiffuLLM ran for 1.2M steps; the SEDD models ran for ≈100k steps (≈8% of that budget at much larger batch sizes). A fair comparison would require SEDD models trained to convergence. The original SEDD paper trains for 1M+ steps with batch size 512 on 8 GPUs - far more compute than available on a single A4000.

8 Conclusion

This project demonstrates that discrete absorbing-state diffusion models can learn to generate domain-specific structured text with high coherence for short-to-medium sequence lengths, and provides a comparison of architectures.

References

- Lou, A., Meng, C., & Ermon, S. (2023). *Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution.* arXiv:2310.16834.
- Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models.* NeurIPS.
- Song, Y., et al. (2021). *Score-Based Generative Modeling through Stochastic Differential Equations.* ICLR.
- Austin, J., et al. (2021). *Structured Denoising Diffusion Models in Discrete State-Spaces.* NeurIPS.
- Anderson, B. D. O. (1982). *Reverse-time diffusion equation models.* Stochastic Processes and their Applications.
- Peebles, W. & Xie, S. (2022). *Scalable Diffusion Models with Transformers.* ICCV.
- Hyvärinen, A. (2005). *Estimation of Non-Normalized Statistical Models by Score Matching.* JMLR.
- Kingma, D. P. & Welling, M. (2013). *Auto-Encoding Variational Bayes.* ICLR.
- Vincent, P. (2011). *A Connection Between Score Matching and Denoising Autoencoders.* Neural Computation.
- Ghazvininejad, M., et al. (2019). *Mask-Predict: Parallel Decoding of Conditional Masked Language Models.* EMNLP.

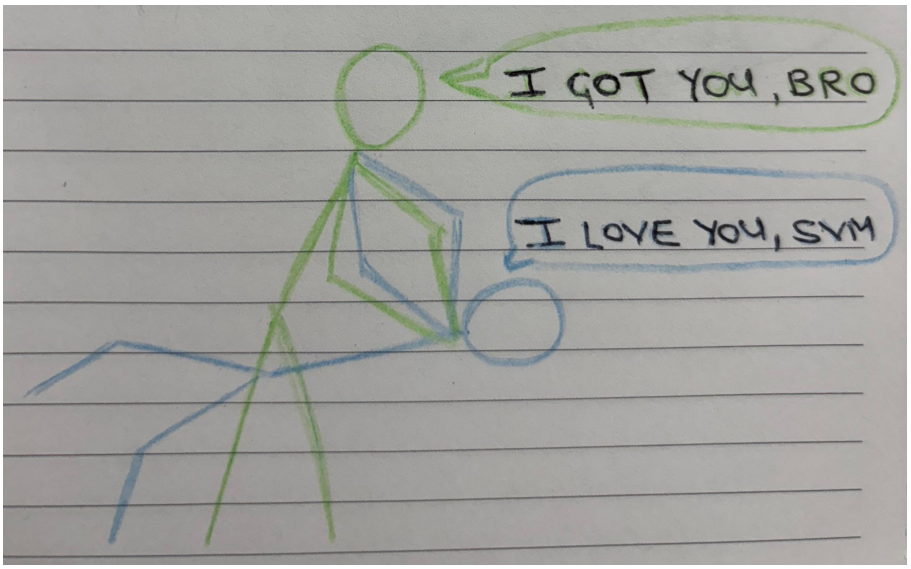


Figure 1: Bonus: We too love Support Vectors



Figure 2: Bonus bonus (not hand drawn, not an ML topic, just a meme we are proud of): Independent and identically distributed random variables